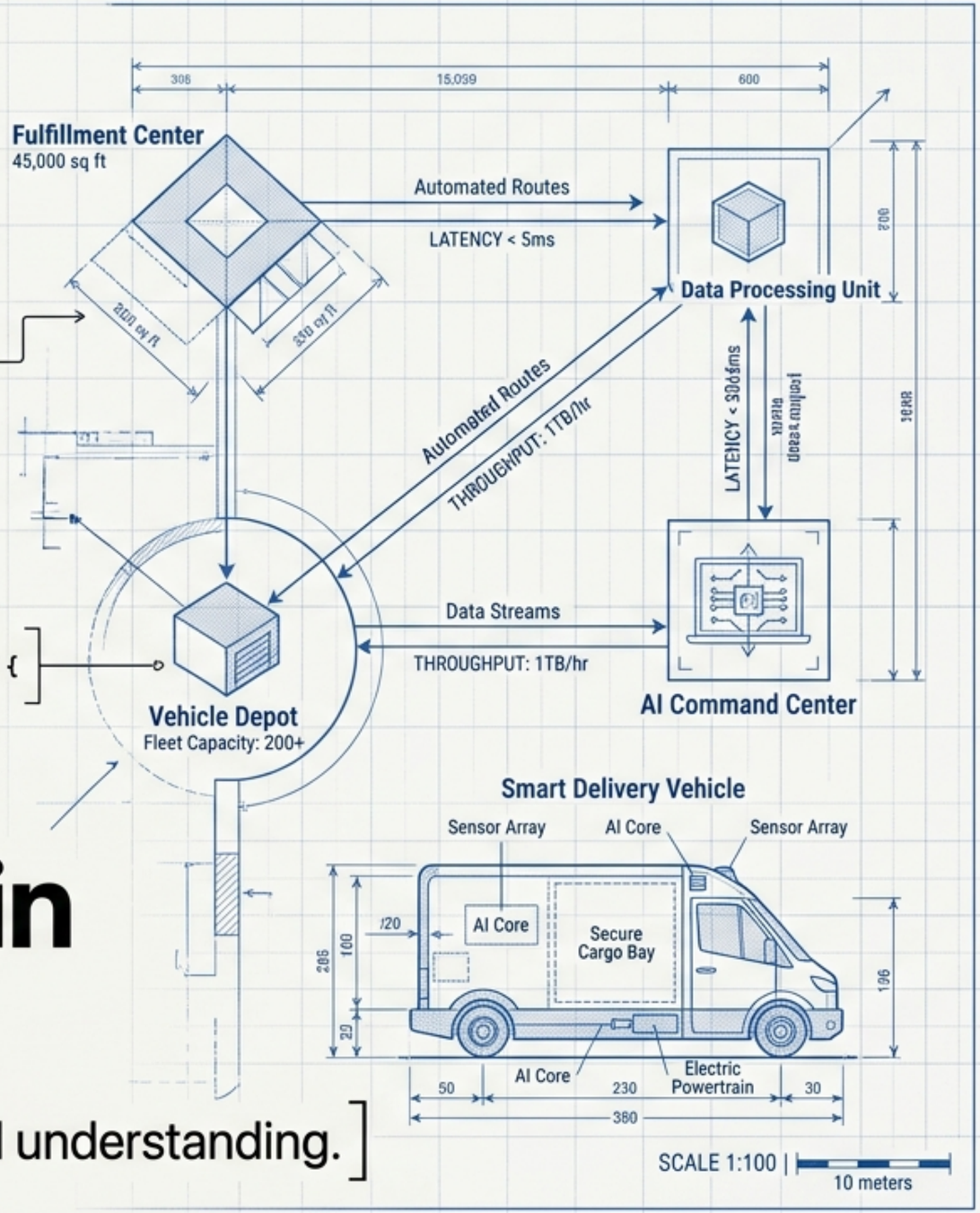
```
// Domain: SmartCity_Logistics
// Entity: Delivery_Vehicle
class DeliveryVehicle {
  constructor(id: String, capacity: Number) {
    this.id = id;
    this.capacity = capacity;
    this.status = "IDLE";
    this.location = new Coordinates(0,0);
  }

  assignRoute(route: Route): void {
    // Pre-condition: Vehicle must be IDLE
    if (this.status != "IDLE") throw new Error("Vehicle unavailable");
    // Action: Assign route and update status
    this.route = route;
    this.status = "EN_ROUTE";
    console.log(`Vehicle ${this.id} assigned route: ${route.id}`);
  }

  updateLocation(newLocation: Coordinates): void {
    // Post-condition: Location updated
    this.location = newLocation;
  }
}

// Rule: Optimize Fleet Utilization
function optimizeFleet(vehicles: DeliveryVehicle[], tasks: DeliveryTask[]): AssignmentPlan {
  // Complex AI-driven optimization logic
  // based on capacity, timing, and traffic.
  // Returns an optimized plan.
  return new AssignmentPlan(/*... */);
}
```



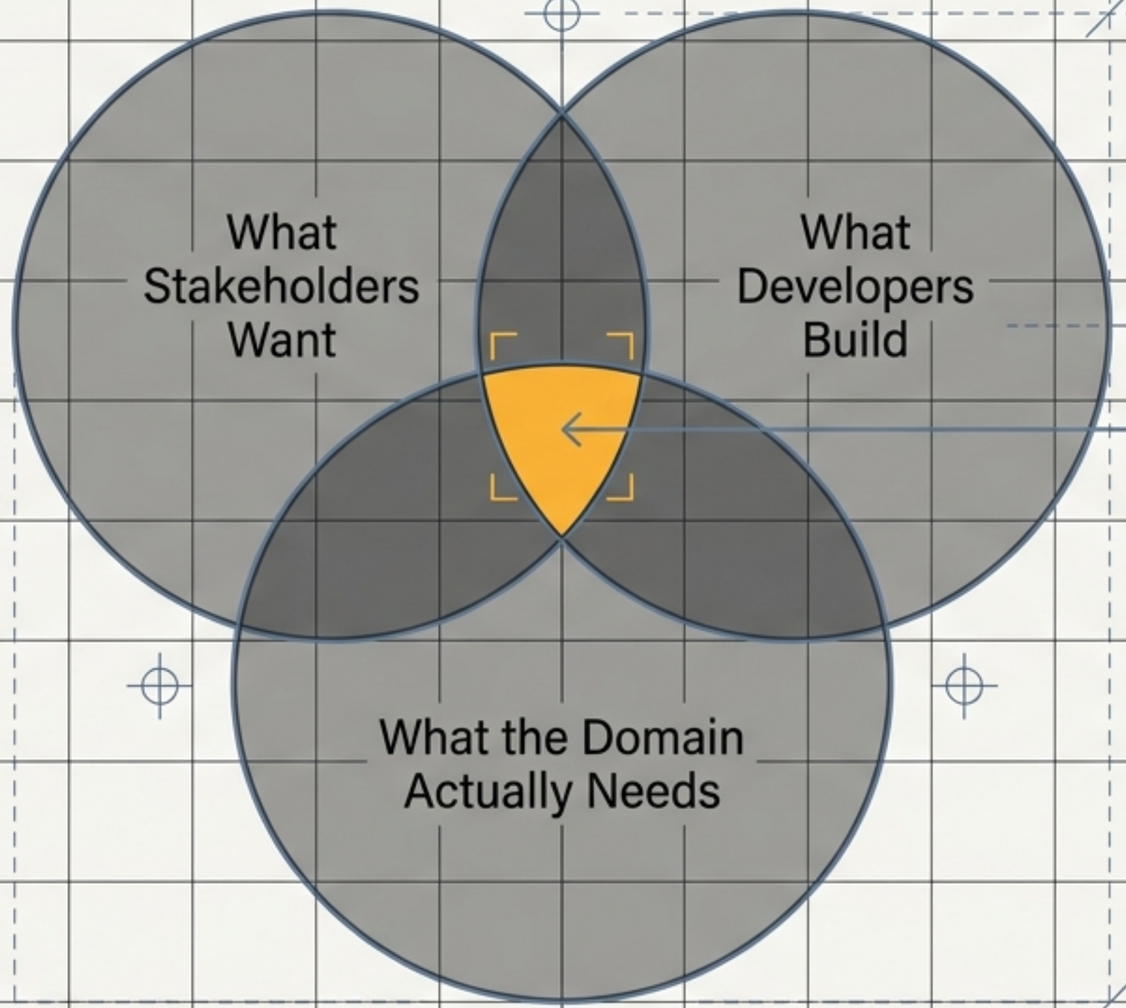
Requirements + Domain Modeling in the AI Era

[Software does not start with code. It starts with shared understanding.]

Gemini Notebook

THE CORE PROBLEM IS NOT DOCUMENTATION

The fundamental problem in software engineering is a lack of shared understanding. As the famous adage goes: “No one knows exactly what they want.”



AI agents fail here. The AI introduces a new layer of miscommunication between human intent and machine execution.

Gemini Notebook

Distinct Phases: Discovery vs. Specification

DISCOVERY	SPECIFICATION
What do we actually need to build?	**What did we agree to build?**
Activities: - Conversations - EventStorming - Probing rules - Finding exceptions	Artifacts: - User Stories - Acceptance Criteria - Domain Models - Tickets
Nature: Unstructured, exploratory	Nature: Structured, rigorous

The ISO/IEC/IEEE 29148 standard defines requirements engineering as a continuous loop of discovering, modeling, and verifying—**not just writing a list.**

Gemini Notebook

THE ANATOMY OF A REAL REQUIREMENT

A requirement is a relevant need or constraint expressed precisely enough to verify it was met.
(Functional = what it does; Non-functional = how it behaves).

The Bad

~~The system must be fast~~

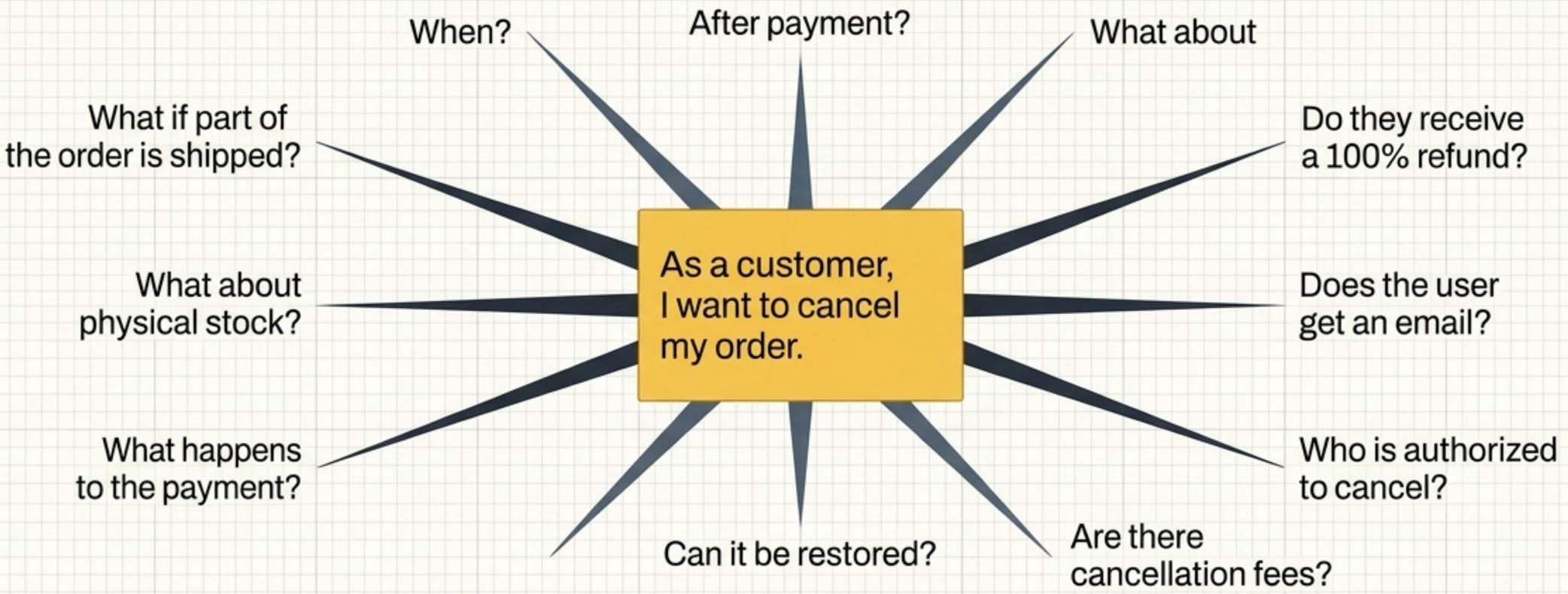
The Good

The search page returns results in <500ms for 95% of requests under a load of 1k req/sec.

- Context
- Metric & Threshold
- Verifiable Condition
- Operating Condition

Gemini Notebook

The Ambiguity Explosion

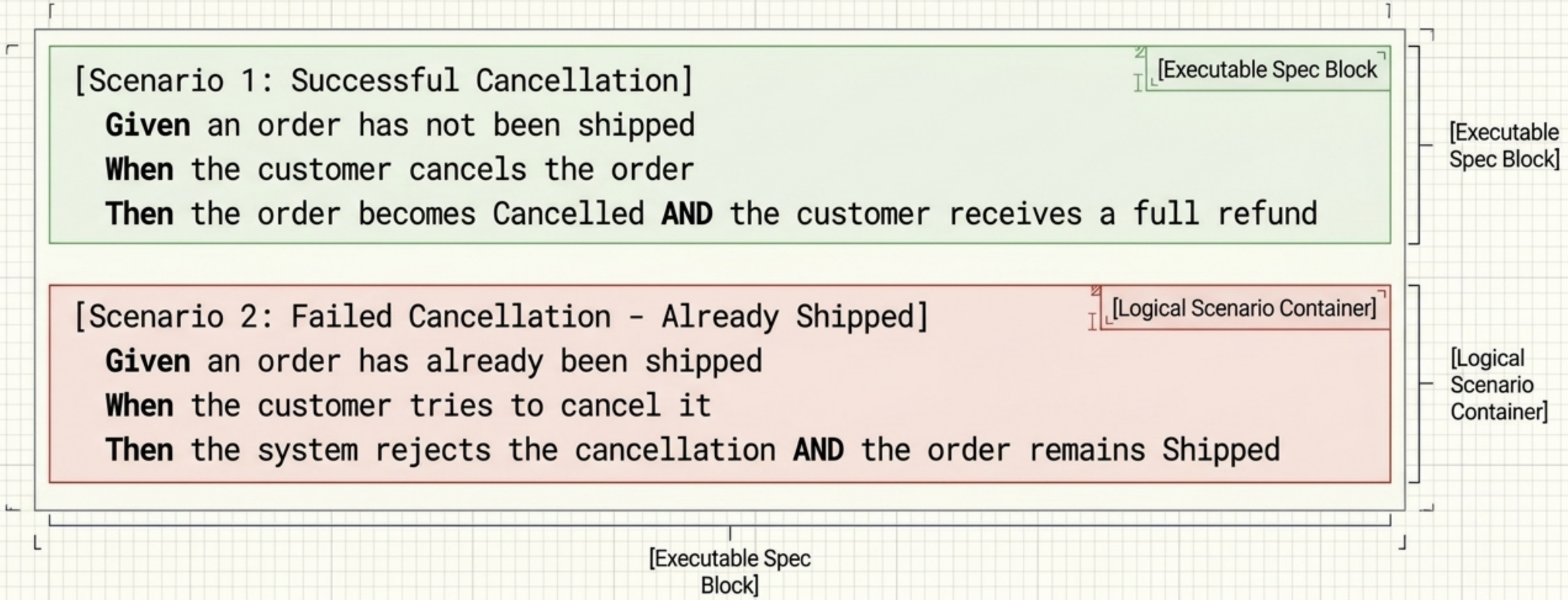


User stories are not complete specifications.
They are invitations to a conversation.

Gemini Notebook

Locking Down Ambiguity with Executable Specs

Acceptance criteria answer the ultimate question:
“How will we know we understood the requirement correctly?”

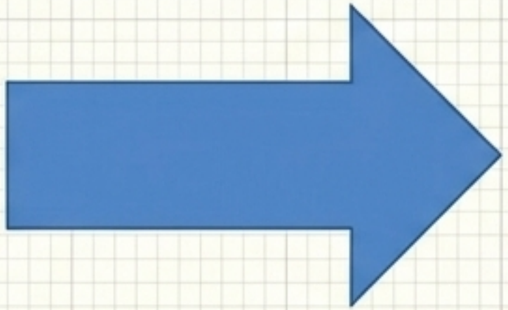


Gemini Notebook

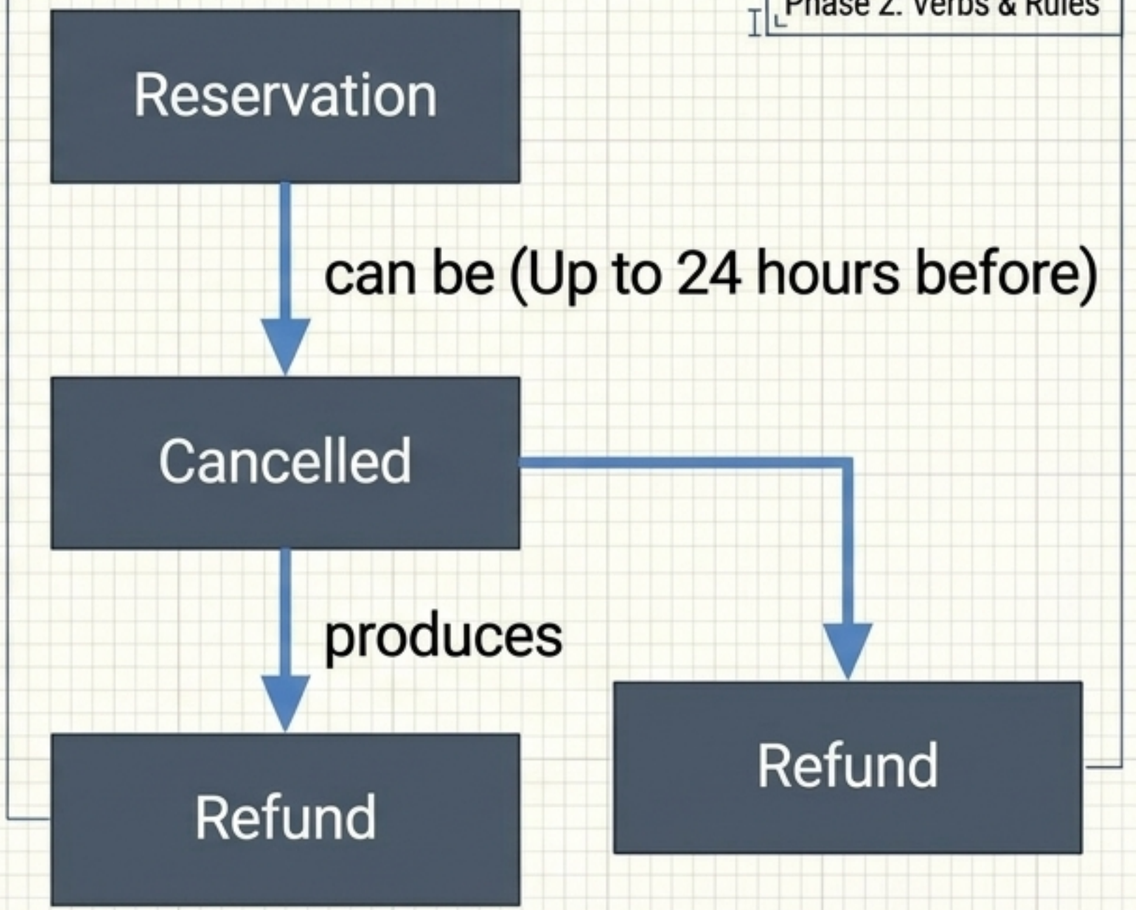
Domain Modeling: Beyond the Nouns

Phase 1: Just Nouns

- Guest
- Property
- Reservation
- Refund



Phase 2: Verbs & Rules

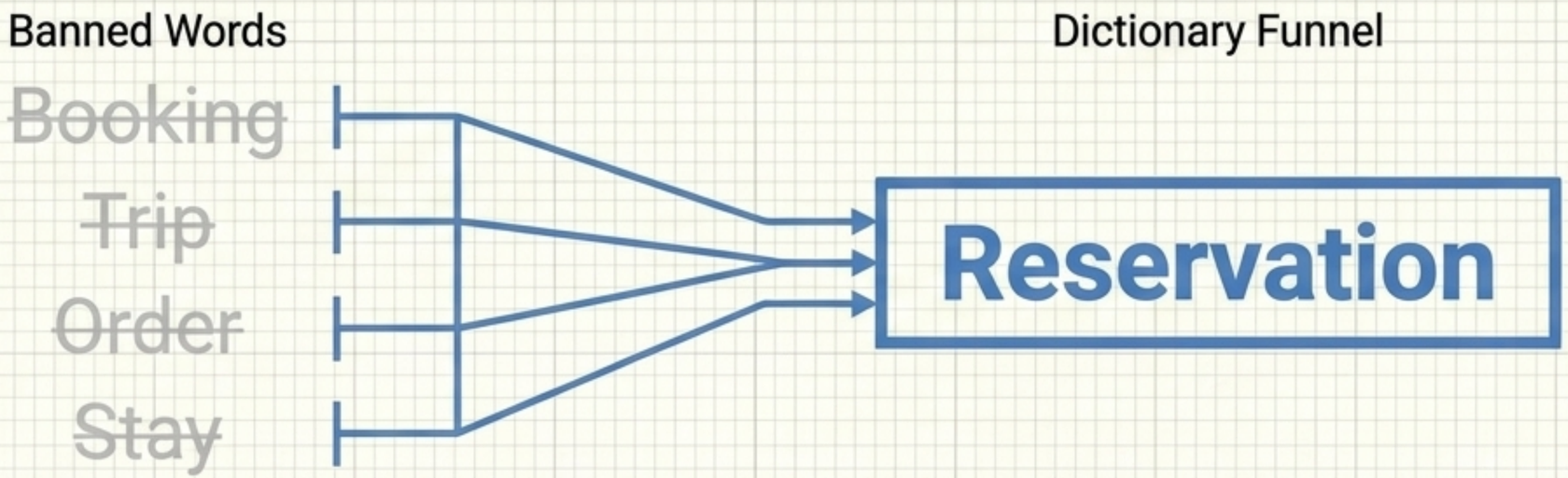


DDD shifts the focus. We stop asking "What classes do we need?" and start asking "What are the rules, concepts, and relationships that dictate behavior?"

Gemini Notebook

Forging a Ubiquitous Language

Developers and domain experts must share a rigorous, canonical language.

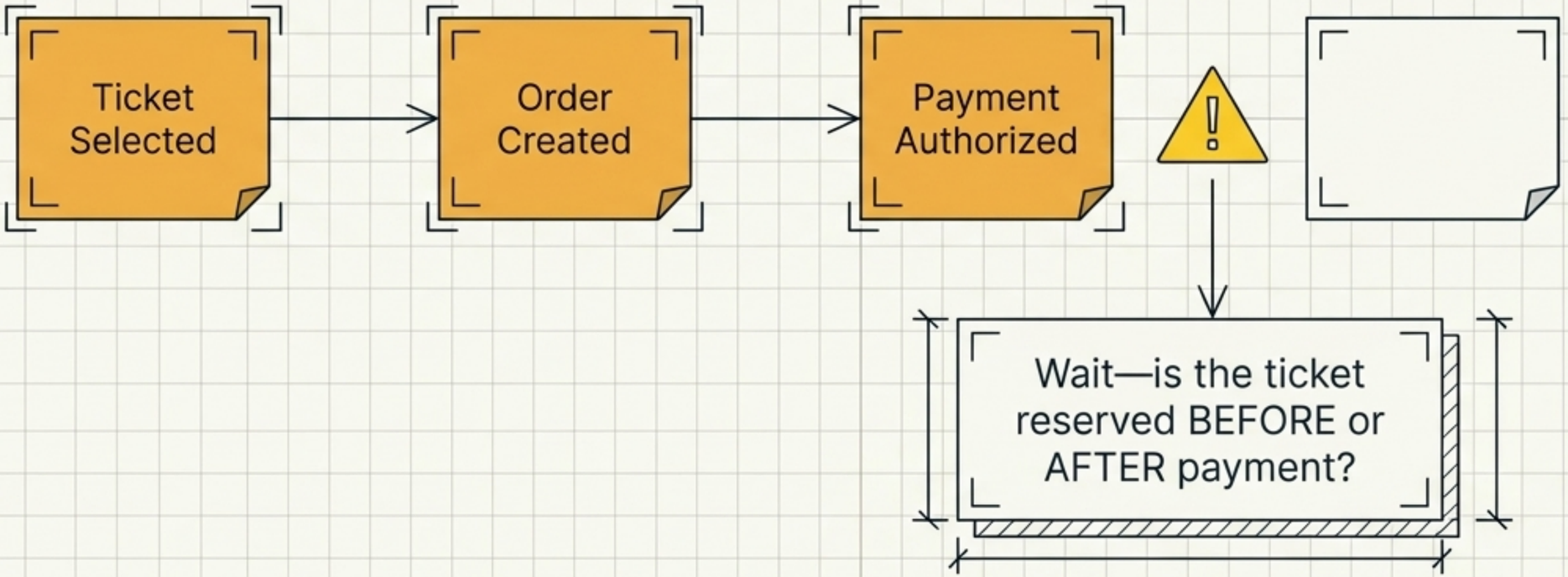


Ubiquitous language isn't just a decorative glossary. It is the exact terminology used in conversations, written into requirements, and baked directly into the codebase (e.g., cancelReservation()).

Gemini Notebook

EventStorming: Visualizing the Domain

We don’t start by asking what entities to build. We ask: "What happened?" Events naturally force the discovery of hidden business policies.



The model is not UML. The goal is the conversation, not a perfect diagram.

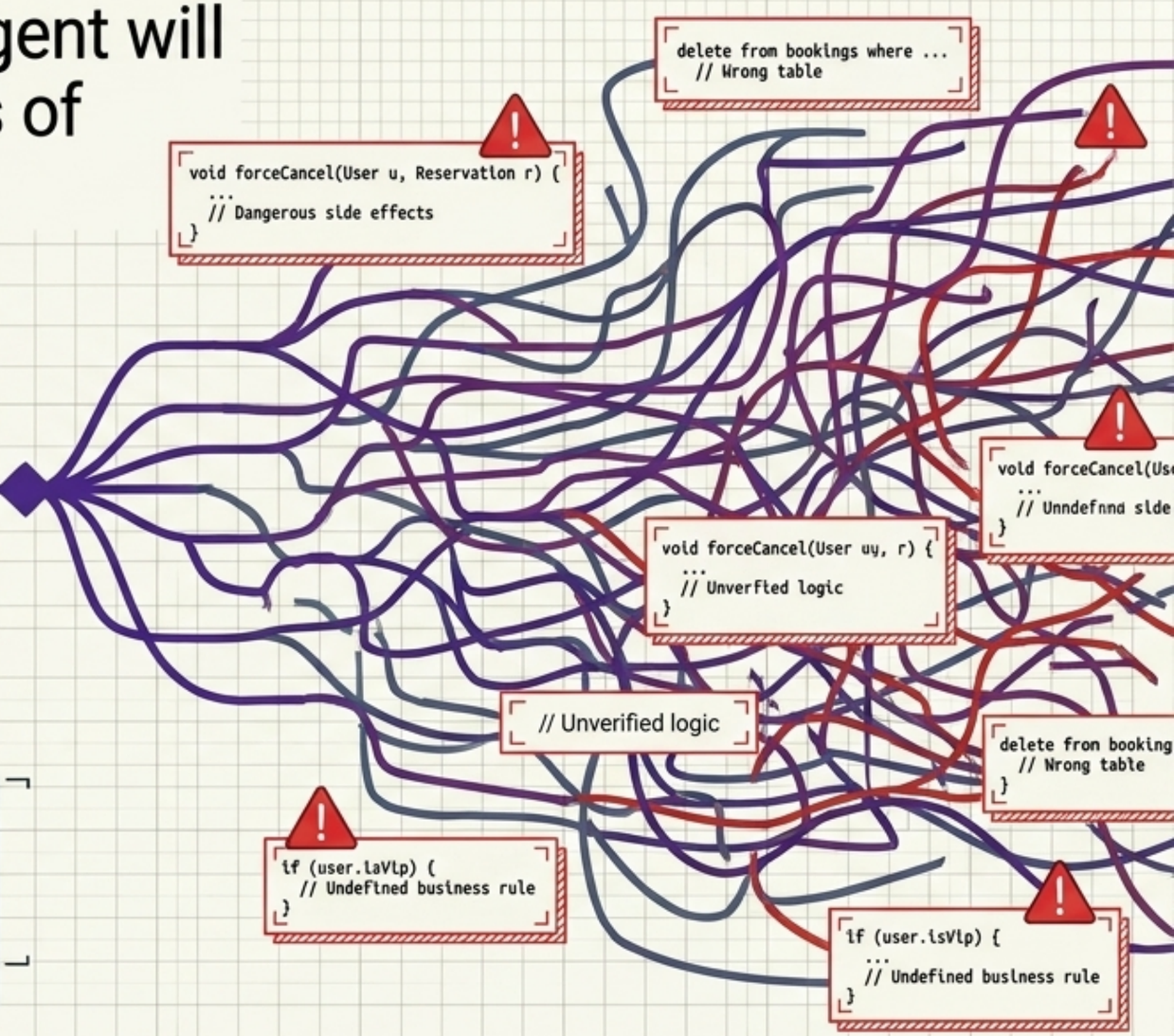
Gemini Notebook

The AI Paradigm Shift: Plausible ≠ Correct

If a requirement is ambiguous, an AI agent will confidently generate massive amounts of the wrong code.

Let users cancel reservations.

AI Agent

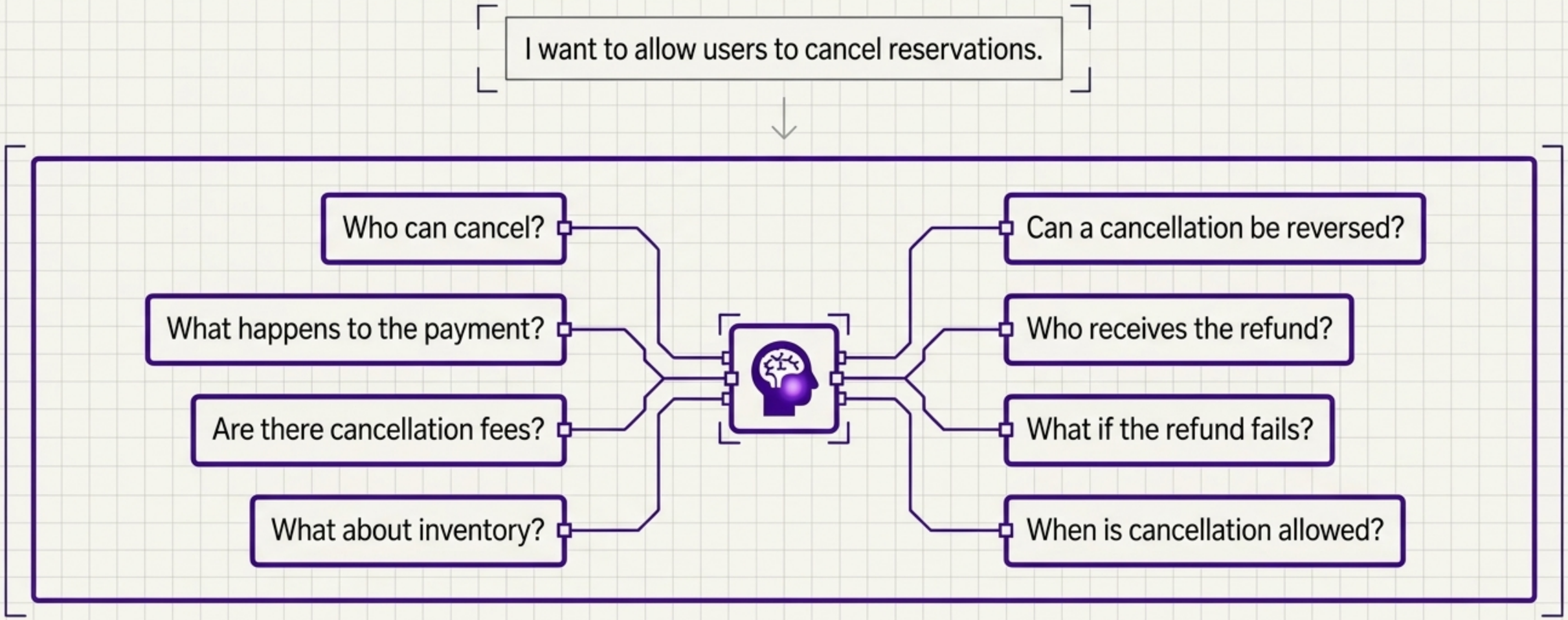


As code generation becomes cheaper, verifying that we are generating the right code becomes the most critical engineering skill.

Gemini Notebook

The “Grill-Me” Reversal

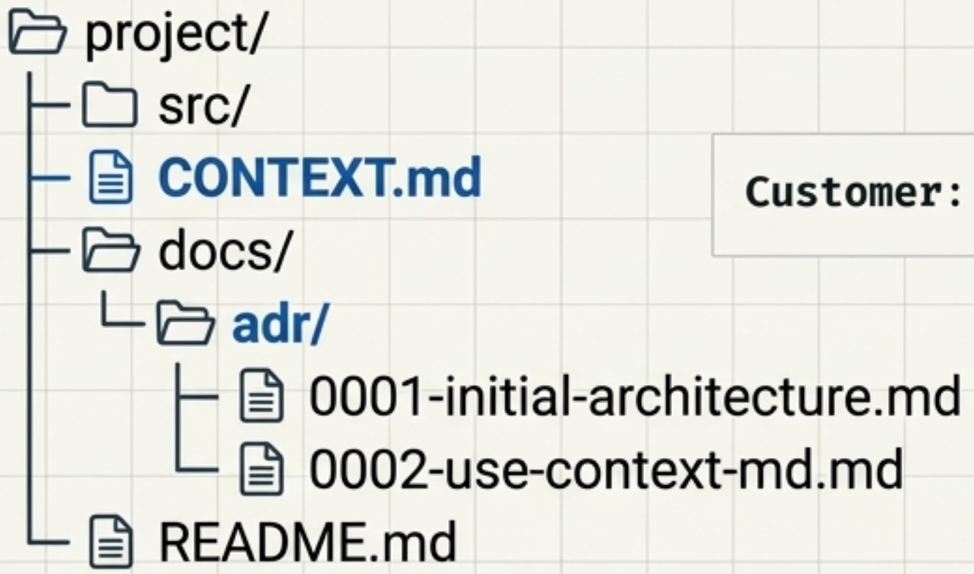
Do not let the agent write code from an idea. Use the agent to discover what actually needs to be specified. Turn “write requirements” into “discover the domain.”



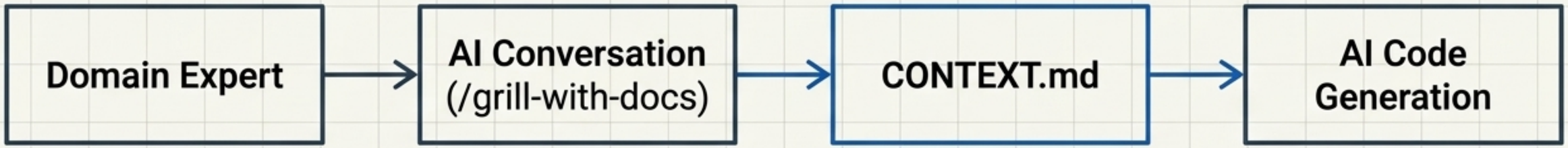
Gemini Notebook

CONTEXT.md: Ubiquitous Language 2.0

The AI automatically registers canonical terms and architectural decision records. When the language is confused, stop the code and model the domain.




```
Customer: A person who places orders. (Avoid: Client, Buyer)
```

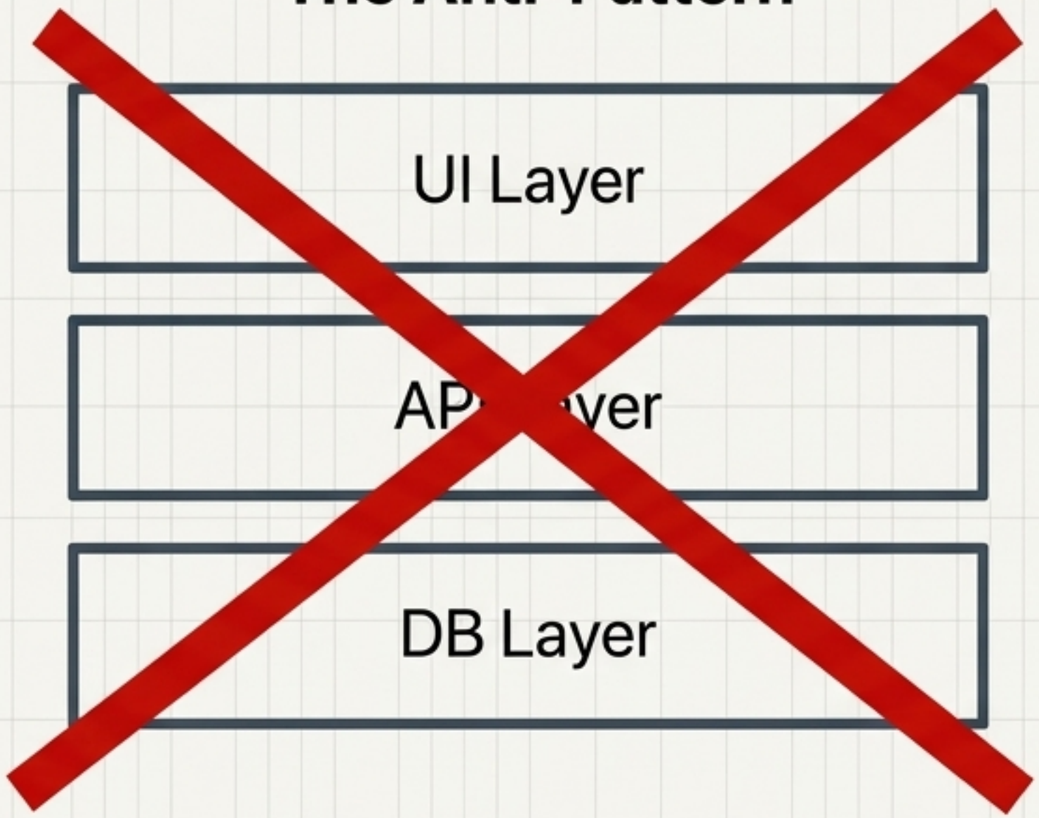


Gemini Notebook

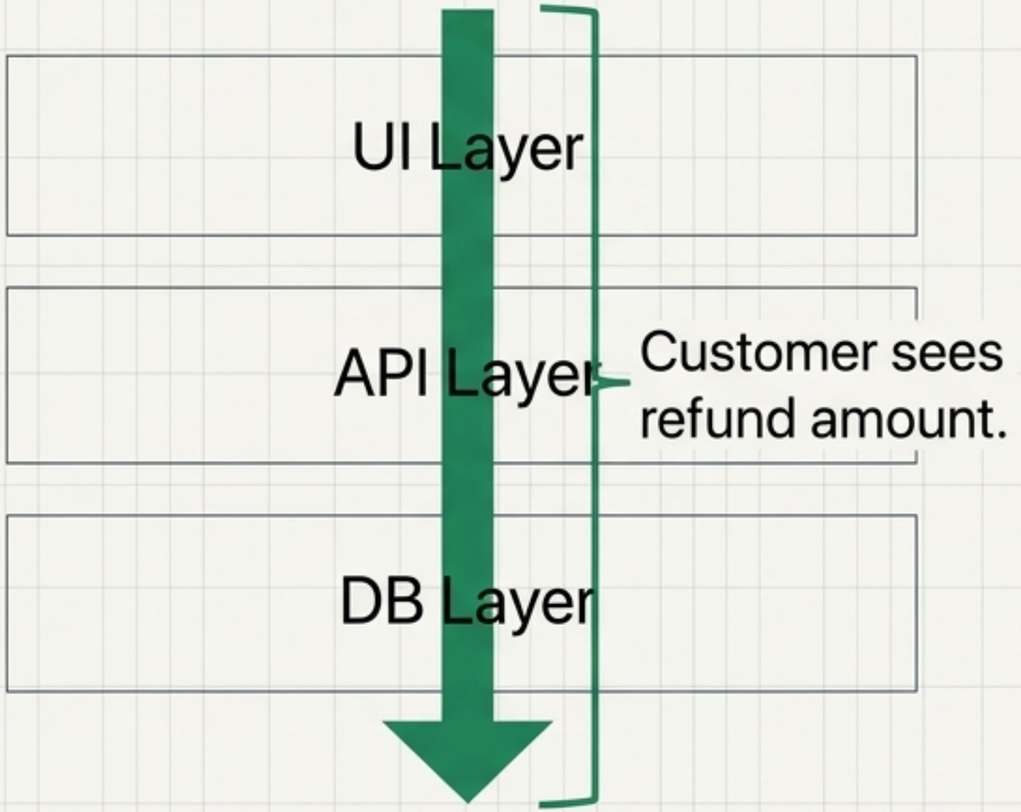
Executing Specifications via Vertical Slices

AI agents shouldn't be given "Frontend" and "Backend" tickets. Specifications must be cut into Tracer Bullets—small, complete paths through all necessary layers to deliver one demonstrable behavior.

The Anti-Pattern



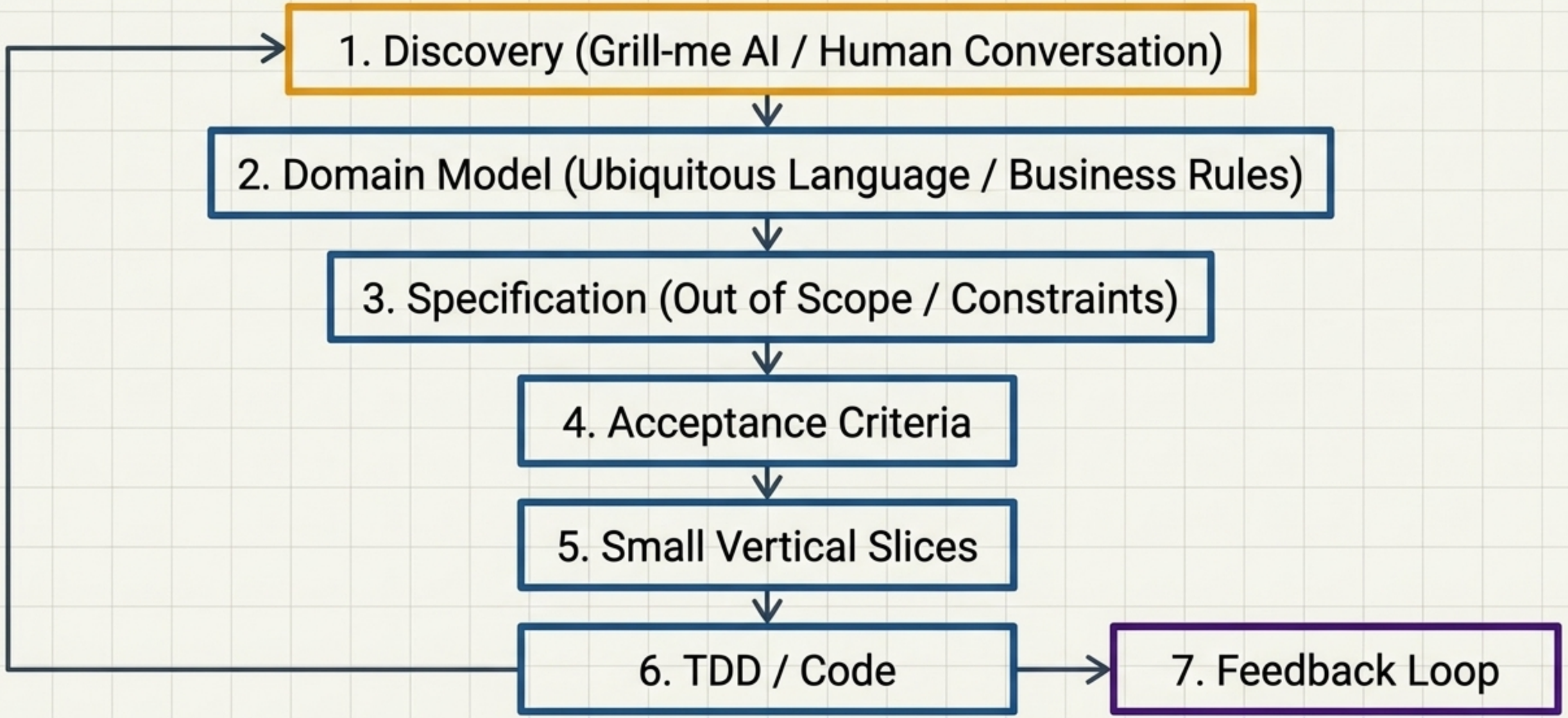
The Tracer Bullet



Gemini Notebook

The Modern Discovery Pipeline

The unified system combining human domain expertise with AI acceleration.



Gemini Notebook

Acceleration vs. Meaning

Software engineering in the AI era is no longer about typing syntax. It is entirely about requirements engineering, domain modeling, and shared understanding.



AI can accelerate every single step of the pipeline.



But AI cannot decide what the the business actually means.

Gemini Notebook